PROYECTO INTERMODULAR



VM BARBERSHOP

Autor: Víctor Molina Trujillo

Tutor: Alejandro Romero Morales

Centro: Cesur Cáceres

Índice

1. Descripción y visión general del proyecto 3

- 1.1 Contexto, objetivo y propósito 3
- 1.2 Arquitectura y ficha de implantación 3
- 1.3 Conclusiones sobre el alcance y resultados esperados 4
- 1.4 Modelo de Roles y Control de Acceso 5

2. Introducción 6

- 2.1 Contexto y problemas detectados 6
- 2.2 Objetivos del sistema 6
  - 2.2.1 Objetivo general 6
  - 2.2.2 Objetivos específicos 6
- 2.3 Alcance y limitaciones 7
  - 2.3.1 Objetivos específicos de desarrollo 7
  - 2.3.2 Limitaciones del sistema (MVP) 7

3. Análisis y planificación 8

- 3.1 Metodología de desarrollo y enfoque del proyecto 8
- 3.2 Introducción al Plan de Sprints e Inventario de Historias de Usuario 8/9
- 3.3 PRIMER SPRINT: Cimentación y Estructura Base 10
  - 3.3.1 Detalle del Sprint 1 10
  - 3.3.2 Configuración del Entorno y Estructura de Archivos (VSC) 10
  - 3.3.3 Diseño y Asentamiento de la Base de Datos 10
  - 3.3.4 Historias de usuario comprobadas en este sprint 10
  - 3.3.5 HU-01: Registro de nuevos clientes 11
  - 3.3.6 HU-02: Consulta del catálogo de servicios 12
  - 3.3.7 HU-09: Autenticación segura y acceso por roles 13
- 3.4 SEGUNDO SPRINT: Lógica de Negocio y Motor de Reservas 14
  - 3.4.1 Desarrollo de la API de Citas 14
  - 3.4.2 Historias de usuario comprobadas en este sprint 15
  - 3.4.3 HU-03: Reserva de cita de forma online 15/16
  - 3.4.4 HU-04: Consulta de citas próximas (Perfil Cliente) 17
  - 3.4.5 HU-07: Consulta de agenda diaria (Rol Barbero) 18
  - 3.4.6 HU-05: Cancelación de citas 19
- 3.5 TERCER SPRINT: Robustez, UX y Validaciones Críticas 20
  - 3.5.1 Lógica de Validación y UX 20
  - 3.5.2 Historias de usuario comprobadas en este sprint 20
  - 3.5.3 HU-06: Resumen de reserva antes de confirmar 20/21
  - 3.5.4 HU-08: Validación de calendario (Pasados y Domingos) 22
  - 3.5.5 HU-10: Filtrado de citas pasadas en la agenda 23

4. Manual de Instalación y Configuración 24

- 4.1 Requisitos del sistema 24
- 4.2 Pasos para la instalación local 24

5. Manual de Usuario (Guía de uso para el cliente) 25-27

6. Conclusiones y Trabajo Futuro 28

- 6.1 Conclusiones del proyecto 28
- 6.2 Trabajo futuro y escalabilidad 28

7. Bibliografía y Recursos 28

1. Descripción y visión general del proyecto

Se presenta el siguiente documento en forma de redacción de la memoria técnica del proyecto que ha recibido el nombre de **“VM-Barbershop”**, el cual se trata de un sistema de gestión de reservas específico para el sector de la estética y peluquería masculina.

1.1 Contexto, objetivo y propósito

VM-Barbershop nace como una solución de software de gestión empresarial orientada a la digitalización de los procesos de reserva dentro del sector de la imagen masculina. El proyecto surge para resolver una problemática crítica detectada en el funcionamiento diario de muchos profesionales: la interrupción constante de la actividad técnica.

Actualmente, una gran parte de los peluqueros gestiona su agenda mediante llamadas telefónicas, lo que les obliga a interrumpir un corte o servicio en curso para atender el teléfono, afectando a la calidad del servicio y a la productividad. A esto se suma el uso de agendas de papel, un método arcaico que presenta graves deficiencias en comparación con un sistema digital: no permite actualizaciones en tiempo real, dificulta la consulta remota por parte del cliente y es propenso a errores humanos o pérdida de datos.

El propósito de esta aplicación es eliminar estas fricciones mediante los siguientes objetivos específicos:

- Automatización de la Agenda: Proporcionar una agenda online actualizable que permita al barbero visualizar su carga de trabajo sin distracciones manuales.
- Optimización del Tiempo: Evitar las interrupciones telefónicas al permitir que sea el propio cliente quien gestione su reserva de forma autónoma a través de la plataforma.
- Centralización de Datos: Sustituir el soporte físico de papel por una base de datos segura y accesible desde cualquier dispositivo, garantizando que la información nunca se pierda y esté siempre al día.

1.2 Arquitectura y ficha de implantación

Para el desarrollo de VM-Barbershop, se ha adoptado un diseño basado en una arquitectura organizada, utilizando el patrón MVC (Modelo-Vista-Controlador). El objetivo de este modelo es separar la lógica de negocio (servidor y datos) de la interfaz de usuario, garantizando así la escalabilidad del proyecto y un mantenimiento mucho más sencillo y organizado.

Para demostrar los conocimientos adquiridos durante el ciclo formativo, se ha seleccionado el siguiente stack tecnológico para la implantación del sistema:

- Lenguaje de programación: El núcleo del proyecto y la lógica del negocio se han desarrollado utilizando JavaScript (ES6+) ejecutado en el entorno de Node.js.

- Servidor y API: Se ha utilizado el framework Express para la creación de un servidor robusto que gestiona las peticiones REST entre el cliente y la base de datos de manera eficiente.

- Base de datos: Para la gestión de datos relacionales se utiliza MySQL 8.0 a través del entorno XAMPP. La comunicación se realiza mediante el driver oficial de MySQL para Node.js, implementando una capa de persistencia que asegura la integridad de citas, usuarios y servicios.

- Interfaz de usuario (Frontend): Se ha optado por un desarrollo "Vanilla" utilizando HTML5, CSS3 y JavaScript, sin dependencias de frameworks externos. Esto garantiza una carga extremadamente rápida y permite un control total sobre el diseño en modo oscuro y la interactividad de la agenda.

- Gestión de dependencias: Se utiliza NPM (Node Package Manager) para la administración de las librerías necesarias para el funcionamiento del servidor y la conexión a la base de datos.

1.3 Conclusiones sobre el alcance del proyecto y resultados esperados

Al concluir el desarrollo, VM-Barbershop se presenta como una herramienta tecnológica estable y eficiente diseñada para operar de manera óptima en un entorno local. El sistema final integra las siguientes funcionalidades y componentes técnicos:

- Estructura de datos especializada: Se ha desarrollado una base de datos relacional (MySQL) configurada específicamente para organizar la información de las citas, los perfiles de clientes y el catálogo de servicios de peluquería.
- Gestión integral del ciclo de reservas: El software incorpora una lógica completa de funciones CRUD, permitiendo que cada registro de cita sea creado, consultado, modificado o eliminado según las necesidades del negocio.
- Sincronización de agendas: Se ha logrado una comunicación fluida entre el servidor y las interfaces, permitiendo que el barbero visualice en tiempo real la disponibilidad de su jornada.
- Validaciones automáticas de calendario: El sistema incluye algoritmos que bloquean la reserva en horarios pasados o en días no laborables, garantizando la coherencia de la agenda.
- Arquitectura API REST: Se ha implementado un servidor basado en Express que expone endpoints específicos para que el frontend pueda consultar la disponibilidad de forma dinámica.
- Garantía de calidad mediante pruebas: El código ha sido verificado a través de una suite de tests que aseguran el correcto funcionamiento de las rutas y la lógica del servidor.

El enfoque del proyecto ha sido desarrollar una solución práctica y alineada con los objetivos del ciclo formativo. Por esta razón, el sistema se centra en la lógica de gestión interna y no incluye, en esta fase, integraciones con plataformas de pago externas reales,

sistemas de facturación complejos ni protocolos de autenticación de alta seguridad, priorizando la creación de un Producto Mínimo Viable (MVP) funcional y profesional.

1.4 Modelo de Roles y Control de Acceso

Para garantizar una gestión eficiente y segura, se ha diseñado el sistema bajo un modelo de **Control de Acceso Basado en Roles (RBAC)**. La aplicación distingue entre dos tipos de entidades con permisos y vistas totalmente diferenciadas:

- **Rol Cliente:** Orientado a la autogestión. Este usuario tiene acceso exclusivo al catálogo de servicios, la consulta de disponibilidad en tiempo real y la gestión de sus propias reservas (creación y cancelación).
- **Rol Barbero (Administrador de Agenda):** Orientado a la operativa técnica. Este rol accede a una interfaz profesional donde se centralizan todas las citas asignadas a su perfil. Su vista está optimizada para la consulta cronológica de la jornada, permitiéndole centrarse en el servicio sin distracciones administrativas.

Esta segmentación no solo mejora la seguridad de los datos, sino que permite que la lógica del servidor (Backend) entregue únicamente la información relevante para cada perfil, optimizando el rendimiento de la plataforma.

2. Introducción

2.1 Contexto y problemas

La digitalización ha pasado de ser una opción a una necesidad básica para la competitividad de los negocios locales. En el sector de las barberías y peluquerías masculinas, he detectado que la gestión de la agenda sigue siendo un cuello de botella que frena la productividad.

Los problemas más recurrentes que he identificado son:

- Interrupciones constantes en el servicio: El profesional debe pausar su trabajo técnico cada vez que recibe una llamada telefónica para anotar una cita, lo que perjudica la atención al cliente actual.
- Fragilidad de la agenda física: El uso de libretas de papel no permite actualizaciones automáticas, es propenso a tachones y errores de lectura, y no ofrece ninguna visibilidad al cliente fuera del horario comercial.
- Ineficiencia en la comunicación: La falta de un sistema centralizado provoca que se pierdan oportunidades de reserva cuando el barbero no puede atender el teléfono, limitando el crecimiento del negocio.

2.2 Objetivos del sistema

El propósito fundamental de este desarrollo es transformar la operativa del negocio, moviéndola hacia un entorno digital donde la gestión del tiempo sea automática y eficiente.

2.2.1 Objetivo general

Diseñar e implementar una plataforma integral de reservas web que elimine la carga administrativa del barbero, permitiendo que la agenda se gestione de forma autónoma por parte de los usuarios.

2.2.2 Objetivos específicos

- Automatización de reservas: Crear un flujo de usuario intuitivo donde el cliente pueda elegir servicio, profesional y hora sin intervención manual.
- Validación de disponibilidad: Desarrollar una lógica de backend en Node.js que impida duplicidades de citas o reservas en días festivos y horarios pasados.
- Optimización del panel profesional: Construir una vista de "AgendaPro" que filtre las tareas del día de forma cronológica y elimine distracciones visuales.
- Persistencia de datos: Garantizar que toda la información se almacene de forma segura en MySQL, sustituyendo definitivamente el soporte físico de papel.

2.3 Alcance y limitaciones

VM-Barbershop ha sido diseñado con un propósito doble: por un lado, solventar una necesidad real de gestión en el sector de la estética masculina y, por otro, cumplir rigurosamente con los objetivos pedagógicos del módulo de "Proyecto Intermodular". El sistema se ha dimensionado para ser totalmente funcional dentro del tiempo de desarrollo disponible.

2.3.1 Objetivos específicos

Para cumplir con las expectativas del ciclo formativo de grado superior, el proyecto abarca los siguientes puntos:

- Aplicación web completa: Entrega de una plataforma funcional desarrollada bajo el entorno de Node.js y Express.
- Funcionalidades de gestión interna: El sistema cubre el ciclo total de la reserva, desde el alta del cliente hasta la visualización en la agenda del barbero y la gestión de servicios.
- Persistencia y consistencia: Diseño de una base de datos MySQL poblada con registros de prueba que demuestra una comunicación fluida y segura con el servidor.
- Interfaz adaptada al usuario: Implementación de vistas dinámicas que permiten al cliente reservar y al barbero organizar su jornada laboral de forma visual.

2.3.2 Limitaciones

Al tratarse de un proyecto académico desarrollado como Producto Mínimo Viable (MVP), existen ciertas fronteras tecnológicas que se han definido para priorizar la estabilidad de las funciones principales:

- Pasarelas de pago: No se incluye una integración real con servicios externos como Stripe o Redsys, aunque el sistema está preparado para recibir estas APIs en futuras fases.
- Entorno de ejecución: La aplicación está configurada para operar en un entorno local (servidor de desarrollo), sin contemplar de momento la complejidad de un despliegue en la nube.
- Seguridad avanzada: Aunque se gestionan perfiles de usuario, el sistema no implementa cifrados de alta complejidad ni protocolos de recuperación de contraseñas por correo en esta versión inicial.

3. Análisis y planificación

3.1 Metodología de desarrollo y enfoque del proyecto

El desarrollo de **VM-Barbershop** se ha gestionado mediante una metodología ágil, permitiendo una construcción incremental del sistema y una adaptación constante a los requisitos técnicos de una aplicación web moderna. Como proyecto de Desarrollo de Aplicaciones Web (DAW), el análisis se ha centrado en garantizar una separación estricta entre la lógica del servidor (Backend) y la interfaz de usuario (Frontend), asegurando que cada componente cumpla con estándares de escalabilidad y eficiencia.

La planificación se ha estructurado en torno al concepto de Historias de Usuario (HU), las cuales definen las necesidades reales del negocio desde la perspectiva del cliente y del barbero profesional. Estas historias han sido el motor de los tres Sprints realizados, permitiendo validar cada funcionalidad de forma independiente antes de su integración final en el sistema completo.

3.2 Introducción al Plan de Sprints

Para lograr un Producto Mínimo Viable (MVP) que resuelva la problemática de la gestión manual de citas, el proyecto se dividió en fases lógicas de desarrollo. Este capítulo detalla cómo se transformaron las necesidades del sector de la peluquería en requisitos técnicos medibles y ejecutables.

A continuación, se presenta el inventario general de Historias de Usuario que definen el alcance total del sistema, para posteriormente desglosar la implementación técnica, las evidencias y las soluciones alcanzadas en cada iteración o Sprint.

ID	ROL	PRIORIDAD	HISTORIA DE USUARIO
HU-01	Cliente	Alta	Como cliente, quiero registrarme en la web para tener una cuenta personal y gestionar mis reservas.
HU-02	Cliente	Alta	Como cliente, quiero ver el catálogo de servicios (precios y tiempos) para saber qué ofrece la barbería.
HU-03	Cliente	Alta	Como cliente, quiero seleccionar un barbero, fecha y hora para reservar una cita de forma online.
HU-04	Cliente	Media	Como cliente, quiero ver mis citas futuras en mi perfil para recordar cuándo debo asistir.
HU-05	Cliente	Media	Como cliente, quiero cancelar mi cita con antelación para que el hueco quede libre para otros.
HU-06	Cliente	Baja	Como cliente, quiero ver un resumen con los datos de mi reserva antes de confirmarla.
HU-07	Barbero	Alta	Como barbero, quiero consultar mi agenda diaria para conocer mis citas y clientes asignados.
HU-08	Cliente	Media	Como cliente, quiero que me impida seleccionar tanto fechas pasadas como los domingos.
HU-09	Todos	Alta	Como usuario, quiero iniciar sesión de forma segura para acceder a las funciones de mi rol.
HU-10	Todos	Media	Como usuario, no quiero que aparezcan en mi agenda citas pasadas.

3.3 PRIMER SPRINT

3.3.1 Detalle del Sprint 1: Cimentación y Estructura Base

En esta iteración inicial, me centré en configurar el entorno de desarrollo y la persistencia de datos. Antes de pasar a la lógica visual, era necesario que el servidor y la base de datos estuvieran perfectamente comunicados.

3.3.2 Configuración del Entorno y Estructura de Archivos (VSC)

El primer paso fue organizar el espacio de trabajo en Visual Studio Code siguiendo el patrón de diseño MVC. Como se observa en la estructura de archivos, decidí separar el proyecto en dos grandes bloques:

- Directorio backend: Donde centralicé toda la lógica del servidor Node.js. Creé el archivo server.js como punto de entrada y la carpeta routes/ para organizar las entidades de manera independiente (usuarios.js, servicios.js, citas.js).

- Directorio frontend: Reservado para los recursos estáticos (HTML, CSS y JS de cliente), manteniendo el código del navegador totalmente independiente del servidor.

- Gestión de dependencias: Inicialicé el proyecto con npm init, lo que generó los archivos package.json y package-lock.json, fundamentales para gestionar librerías como Express y el conector de MySQL.

3.3.3 Diseño y Asentamiento de la Base de Datos

La base de datos es el corazón de la aplicación. Utilizando MySQL, diseñé un esquema relacional que permitiera cumplir con las historias de usuario de este sprint:

- Tabla usuarios: Diseñada para almacenar credenciales, nombres y el rol (cliente o barbero), lo que permite al sistema identificar qué funciones mostrar tras el login.

- Tabla servicios: Creada con campos para el nombre del servicio, el precio y la duración estimada, datos críticos para que el cliente pueda consultar el catálogo.

- Conectividad: Desarrollé el archivo db.js, donde configuré el *pool* de conexiones, permitiendo que el backend realice consultas SQL de forma segura y eficiente.

3.3.4 Historias de usuario comprobadas en este sprint

En esta fase, el objetivo principal fue la **validación y el control de calidad** de las funcionalidades desplegadas en la interfaz de usuario. No solo se comprobó que el código ejecutara las tareas, sino que la experiencia del usuario fuera coherente, segura y libre de errores lógicos.

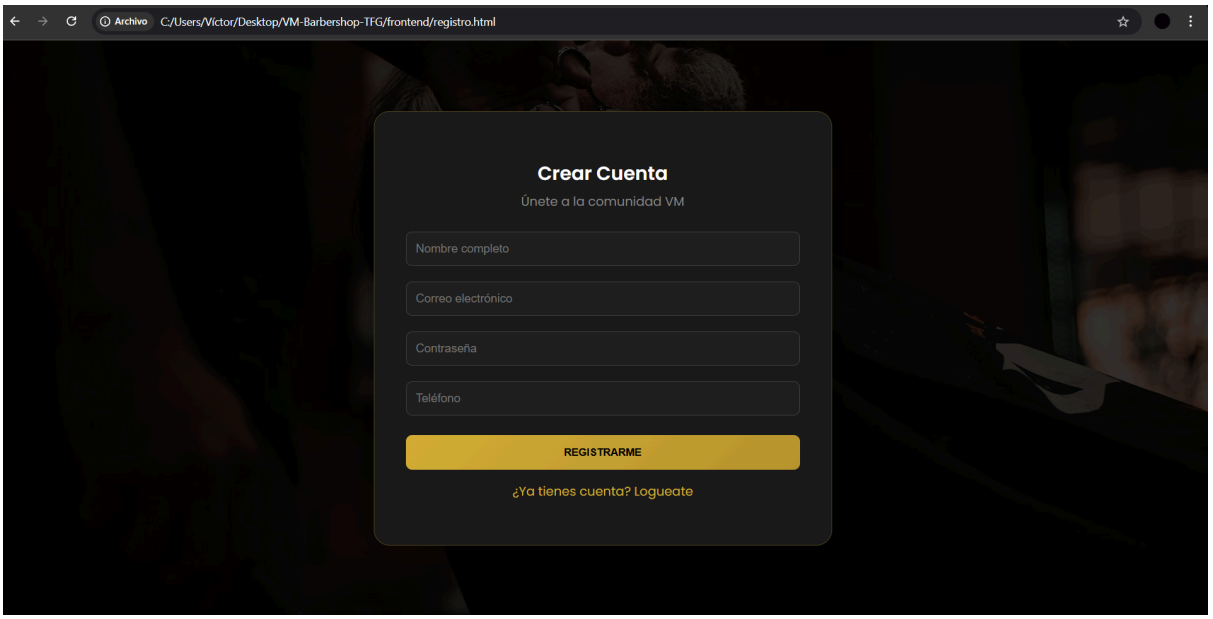
A continuación, se detallan las Historias de Usuario verificadas y los criterios de aceptación cumplidos:

ID	ROL	HISTORIA DE USUARIO	ESTADO
HU-01	Cliente	Como cliente, quiero registrarme en la web para tener una cuenta personal y gestionar mis reservas.	COMPROBADO
HU-02	Cliente	Como cliente, quiero ver el catálogo de servicios (precios y tiempos) para saber qué ofrece la barbería.	COMPROBADO
HU-09	Todos	Como usuario, quiero iniciar sesión de forma segura para acceder a las funciones de mi rol.	COMPROBADO

3.3.5 Historia de Usuario 1: Registro de nuevos clientes

El primer paso para digitalizar la barbería es permitir que el usuario pase de ser un visitante anónimo a un cliente registrado. Esta funcionalidad es la puerta de entrada al sistema y permite que cada reserva quede vinculada a una identidad única en la base de datos.

A. Implementación de la Interfaz (Frontend) Para cumplir con este requisito, se ha diseñado un formulario de registro en el archivo registro.html. El objetivo ha sido crear una interfaz minimalista y funcional que recoja los datos estrictamente necesarios: nombre, apellidos, teléfono, correo electrónico y contraseña. Se han aplicado validaciones en el lado del cliente para asegurar la integridad de los campos antes de ser enviados al servidor.



B. Lógica de Persistencia y Servidor (Backend)

La verdadera potencia de esta historia de usuario reside en el backend. He desarrollado un endpoint de tipo **POST** en el archivo routes/usuarios.js. Este controlador se encarga de recibir los datos mediante el objeto req.body, establecer una conexión con MySQL a través del pool configurado en db.js y ejecutar una sentencia INSERT INTO para persistir al nuevo cliente en el sistema.

```
    <title>Login - VM Barbershop</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <form id="formLogin">
            <h2>VM Barbershop</h2>
            <p style="text-align: center; margin-top: -15px; color: #888;">Inicia sesión para reservar</p>

            <input type="email" id="email" placeholder="Correo electrónico" required>
            <input type="password" id="password" placeholder="Contraseña" required>

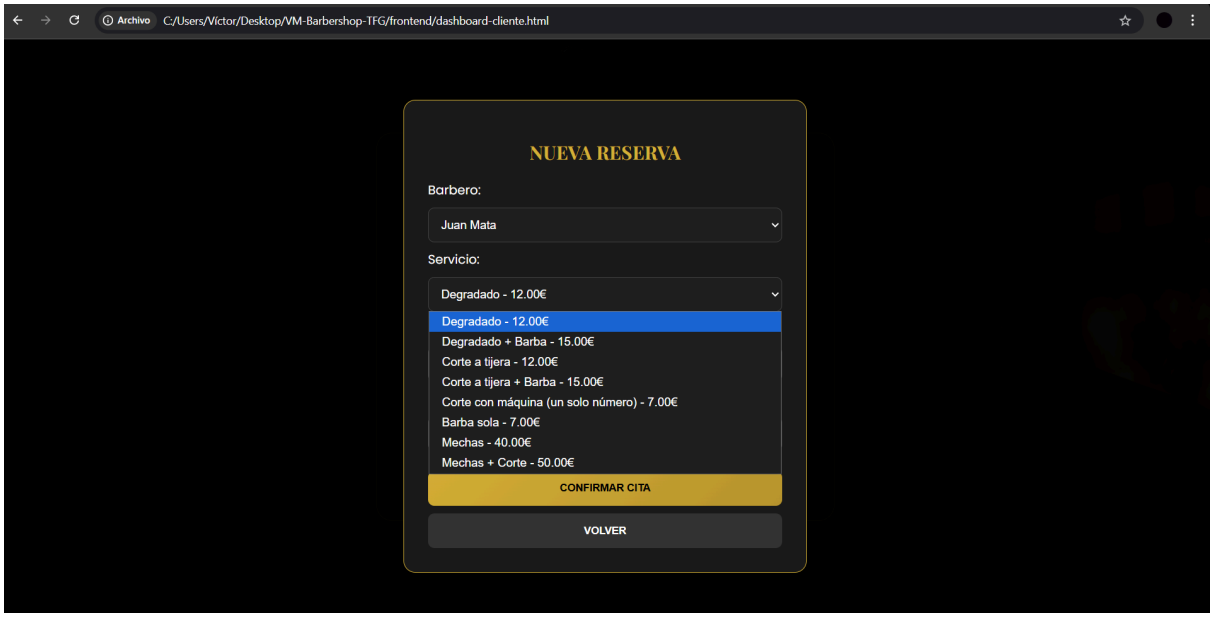
            <button type="submit">ENTRAR</button>

            <p style="font-size: 0.9em; text-align: center; color: #666; margin-top: 15px;">
                ¿No tienes cuenta? <a href="registro.html" style="color: #d4af37; text-decoration: none; font-weight: bold;">¡Regístrate ya!</a>
```

3.3.6 Historia de Usuario 2: Consulta del catálogo de servicios

Una vez que el usuario accede a la plataforma, es vital que pueda conocer la oferta comercial de la barbería de manera clara y actualizada. Esta funcionalidad elimina la necesidad de consultas externas y permite al cliente tomar una decisión informada antes de proceder con la reserva.

A. Implementación de la Interfaz (Frontend) Se ha desarrollado una sección dentro del dashboard donde se listan de forma visual los servicios disponibles. La interfaz presenta cada servicio con su nombre, precio y duración estimada (tiempo). Al ser una carga dinámica, si el administrador cambia un precio en la base de datos, este se actualiza automáticamente para todos los clientes.



B. Lógica de Consulta y API (Backend)
Para dar soporte a esta vista, el servidor implementa una ruta de tipo **GET** en el archivo routes/servicios.js. La lógica se basa en realizar una consulta SELECT a la tabla servicios de MySQL, devolviendo un objeto JSON que el frontend procesa y renderiza. Esta separación asegura que la información siempre sea veraz y esté centralizada.

backend > routes > servicios.js > ...

```
const express = require('express');
const router = express.Router();
const db = require('../db');

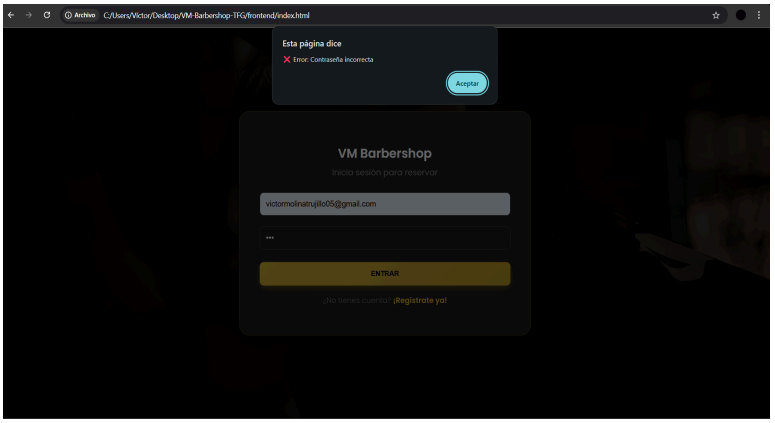
// Ruta para obtener todos los servicios ordenados por prioridad
router.get('/', async (req, res) => {
    try {
        // Añadimos 'ORDER BY orden ASC' para que aparezcan en tu orden: 1, 2, 3...
        const [servicios] = await db.query('SELECT * FROM servicios ORDER BY orden ASC');
        res.json(servicios);
    } catch (error) {
        console.error(error);
        res.status(500).send('Error al obtener servicios');
    }
});

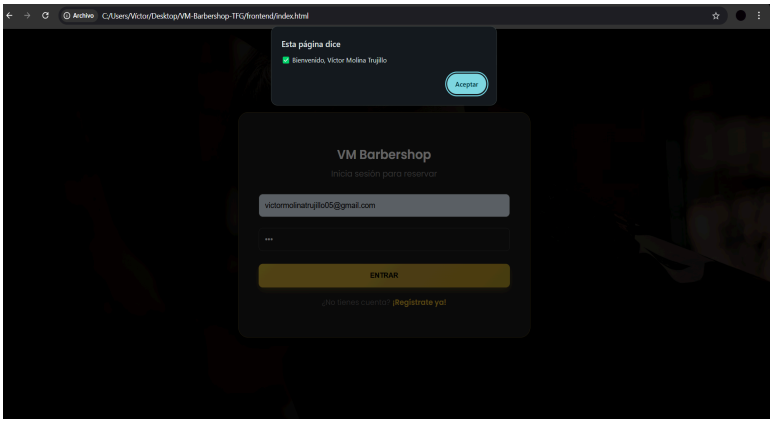
module.exports = router;
```

3.3.7 Historia de Usuario 9: Autenticación segura y acceso por roles

Esta funcionalidad es el núcleo de seguridad de **VM-Barbershop**. No solo garantiza que el acceso esté restringido a usuarios registrados, sino que actúa como un "filtro" que dirige a cada usuario a su entorno de trabajo específico según su rango en la base de datos.

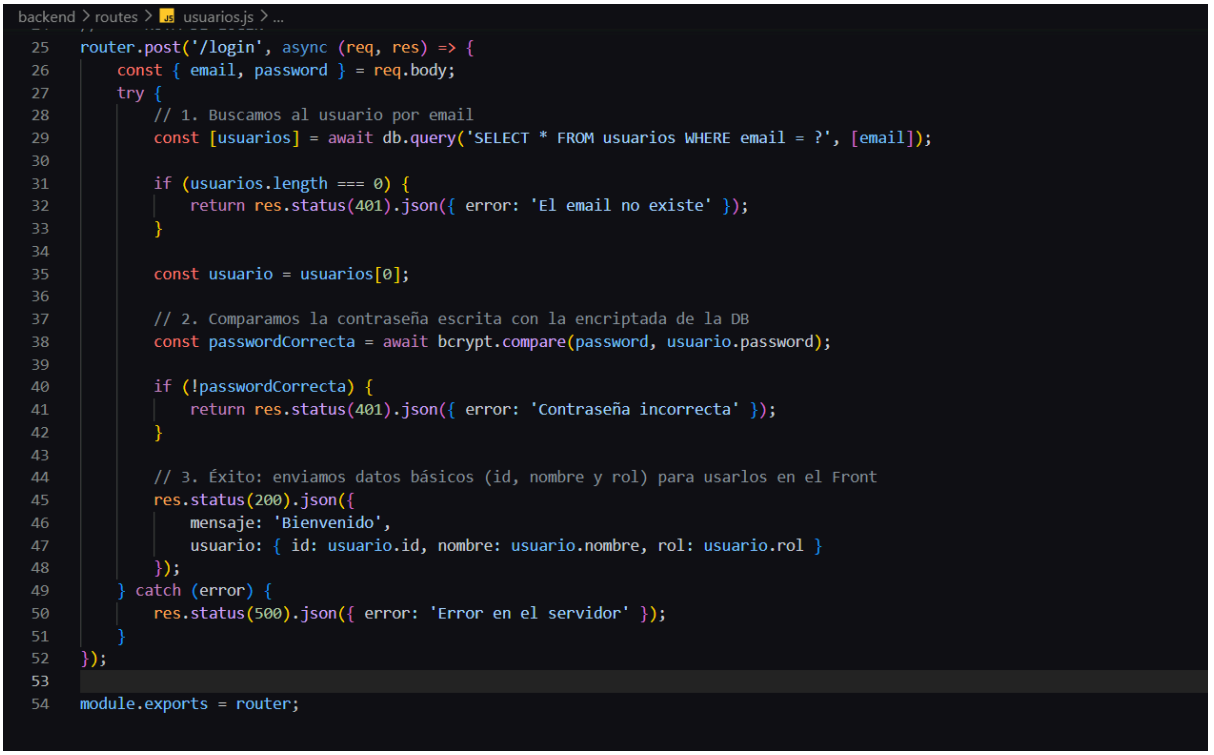
A. Implementación de la Interfaz (Frontend) El acceso se centraliza en login.html. Tras introducir las credenciales, el script login.js procesa la respuesta del servidor. Si la autenticación es exitosa, el sistema lee el campo rol del JSON devuelto y redirige automáticamente: los clientes son enviados a su panel de reservas, mientras que los usuarios con rol "barbero" acceden directamente a la gestión de la agenda profesional.





B. Lógica de Verificación y Cifrado (Backend) La robustez de este proceso reside en el controlador router.post('/login') que se encuentra en el servidor. El proceso técnico sigue un flujo de seguridad estricto:

1. **Validación de existencia**: Se busca el email en la tabla usuarios de MySQL.
2. **Criptografía asíncrona**: Se utiliza la función bcrypt.compare() para verificar la contraseña. Al estar encriptada en la base de datos, este método es el único capaz de validar la autenticidad sin exponer el dato real.
3. **Autorización**: Una vez validado, el servidor envía los datos básicos (ID, nombre y rol), permitiendo que el frontend mantenga una sesión coherente con los permisos del usuario.



3.4 SEGUNDO SPRINT

En esta iteración, el desarrollo se centró en la creación del motor de reservas y la persistencia de las agendas. El reto técnico fue asegurar que la información fluyera correctamente entre los distintos roles (cliente y barbero) garantizando la integridad de los datos en todo momento.

3.4.1 Desarrollo de la API de Citas

Para soportar las nuevas funcionalidades, se creó el archivo routes/citas.js, donde se implementó la lógica necesaria para interactuar con la tabla citas de MySQL:

- Consultas multitabla (JOINs): Se desarrollaron queries para que el cliente pudiera ver no solo el ID de su cita, sino también el nombre del barbero y el servicio asociado, combinando datos de varias tablas.
- Control de estados: Se implementó la lógica para que una cita pueda ser creada, consultada o eliminada (cancelación), cumpliendo con el ciclo de vida completo de una reserva.

3.4.2 Historias de usuario comprobadas en este sprint

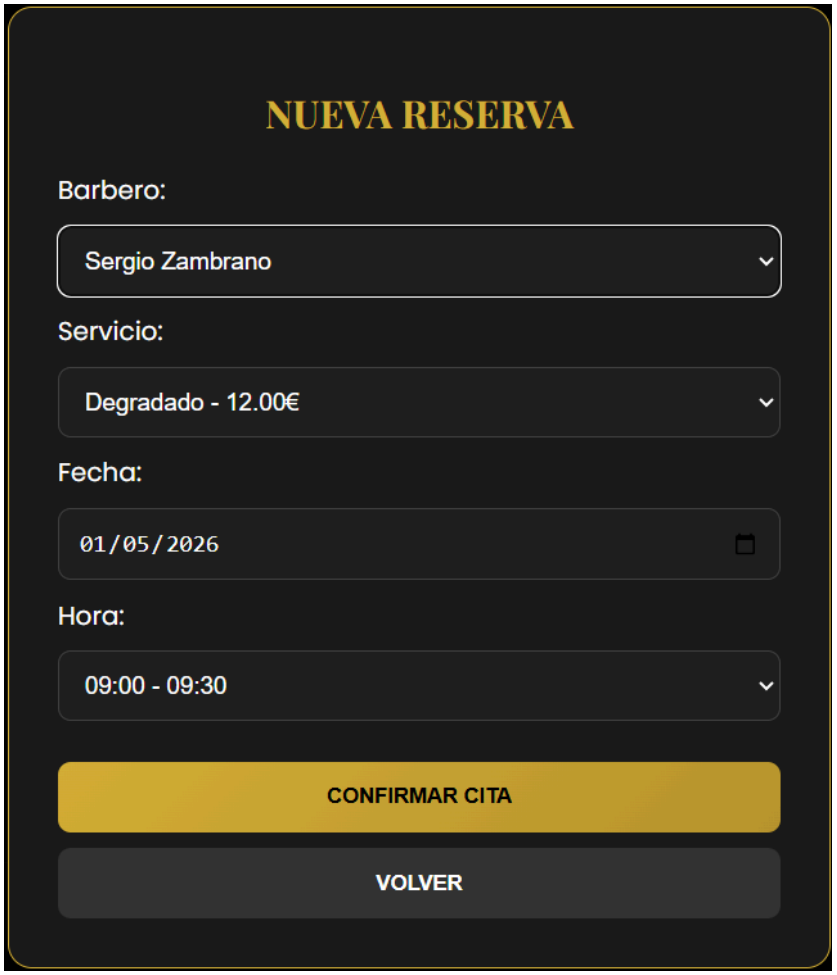
ID	ROL	HISTORIA DE USUARIO	ESTADO
HU-03	Cliente	Como cliente, quiero seleccionar un barbero, fecha y hora para reservar una cita de forma online.	COMPROBADO
HU-04	Cliente	Como cliente, quiero ver mis citas futuras en mi perfil para recordar cuándo debo asistir.	COMPROBADO
HU-07	Barbero	Como barbero, quiero consultar mi agenda diaria para conocer mis citas y clientes asignados.	COMPROBADO
HU-05	Cliente	Como cliente, quiero cancelar mi cita con antelación para que el hueco quede libre para otros.	

3.4.3 Historia de Usuario 3: Reserva de cita de forma online

Esta funcionalidad representa el valor principal de la aplicación, permitiendo que el flujo de trabajo de la barbería se automatice por completo. El objetivo técnico es capturar una intención de reserva y transformarla en un registro persistente y válido en el sistema.

A. Implementación de la Interfaz (Frontend)

El usuario interactúa con un formulario dinámico donde, tras haber iniciado sesión, puede seleccionar tres parámetros críticos: el profesional (barbero), el servicio deseado y el bloque horario disponible. La interfaz utiliza peticiones fetch para enviar estos datos al servidor en formato JSON, asegurando una experiencia fluida sin recargas de página.



B. Lógica de Negocio y Transaccionalidad (Backend)

La gestión de esta historia de usuario se encuentra en el controlador POST del archivo routes/citas.js. El proceso técnico es el siguiente:

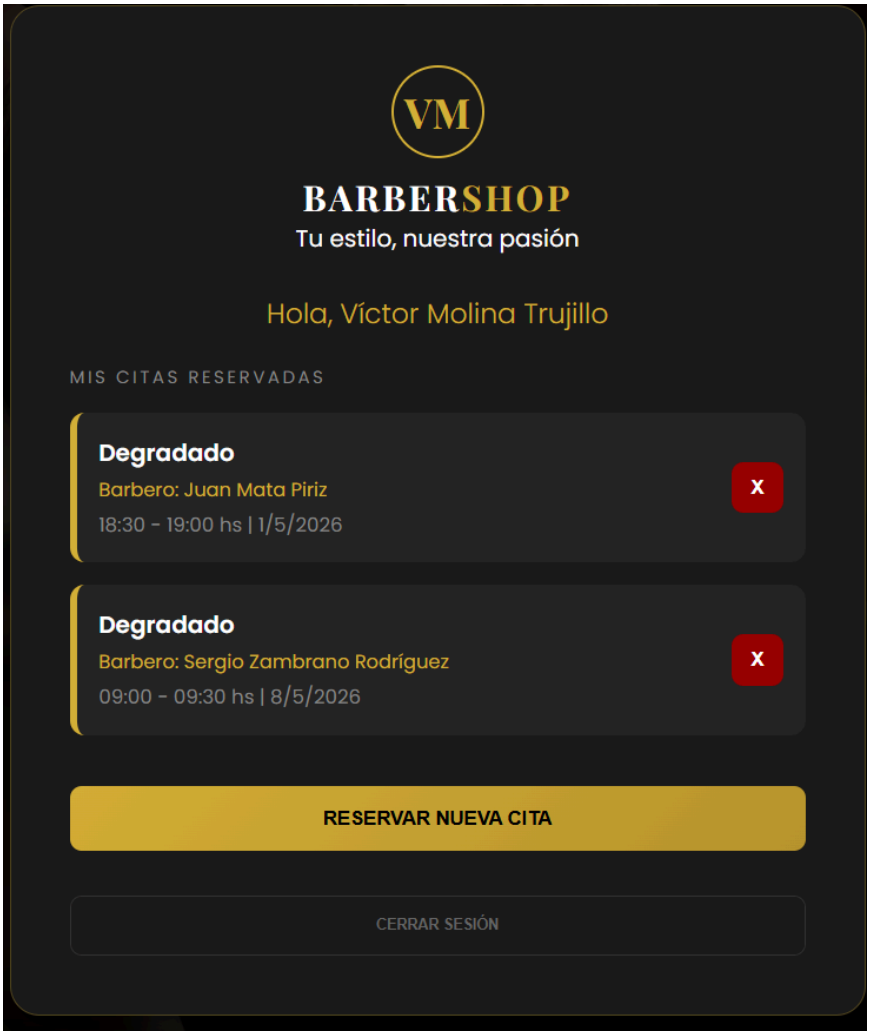
1. Recepción de datos: Se extraen del cuerpo de la petición (req.body) los IDs del cliente, barbero y servicio, además de la fecha y hora.
2. Persistencia Relacional: Se ejecuta una sentencia INSERT INTO citas, donde se relacionan las claves foráneas de las tablas de usuarios y servicios.
3. Confirmación: El servidor responde con un código de estado 201 (Created), lo que confirma al cliente que su espacio en la agenda ha sido reservado con éxito.

```
// 2. GUARDAR RESERVA
router.post('/reservar', async (req, res) => {
    const { usuario_id, servicio_id, barbero_id, fecha, hora } = req.body;
    try {
        const query = 'INSERT INTO citas (cliente_id, barbero_id, servicio_id, fecha, hora, estado) VALUES (?, ?, ?, ?, ?, "pendiente")';
        await db.query(query, [usuario_id, barbero_id, servicio_id, fecha, hora]);
        res.json({ mensaje: '✅ Cita guardada correctamente' });
    } catch (error) {
        res.status(500).json({ error: 'Error al guardar la cita' });
    }
});
```

3.4.4 Historia de Usuario 4: Consulta de citas próximas (Perfil Cliente)

El objetivo de esta funcionalidad es proporcionar al cliente un panel de control personal donde pueda gestionar sus tiempos y recordar sus compromisos con la barbería sin necesidad de contactar telefónicamente.

A. Implementación de la Interfaz (Frontend) En el área personal del cliente, se ha desarrollado una tabla dinámica que consume los datos de la API en tiempo real. La interfaz utiliza JavaScript para iterar sobre el JSON recibido y generar filas de tabla que muestran de forma clara: la fecha, la hora, el nombre del barbero y el servicio contratado. Se ha priorizado la legibilidad mediante un diseño limpio y una ordenación cronológica.



B. Lógica de Negocio y Consultas Complejas (Backend) La implementación técnica se localiza en la ruta GET /proxima/:clienteId del archivo citas.js. Desde el punto de vista de un desarrollador web, destacan los siguientes aspectos:

- Relación de Tablas (JOINs): Se realiza una consulta que vincula las tablas citas, servicios (para obtener el nombre del tratamiento) y usuarios (para obtener el

nombre del barbero), evitando mostrar IDs internos que no aportarían valor al cliente.

- Cláusula de Filtrado Temporal: Se utiliza la instrucción SQL c.fecha >= CURDATE(), lo que garantiza que el servidor solo devuelva citas vigentes, optimizando la carga de datos y la experiencia de usuario.
- Parámetros de Ruta: El uso de :clienteId permite que el endpoint sea dinámico y seguro, recuperando únicamente la información perteneciente al usuario que ha iniciado sesión.

```
// 4. VER CITAS DEL CLIENTE (Panel Cliente)
router.get('/proxima/:clienteId', async (req, res) => {
    try {
        const query = `
            SELECT c.id, c.fecha, c.hora, s.nombre as servicio, u.nombre as barbero
            FROM citas c
            JOIN servicios s ON c.servicio_id = s.id
            JOIN usuarios u ON c.barbero_id = u.id
            WHERE c.cliente_id = ? AND c.fecha >= CURDATE()
            ORDER BY c.fecha ASC, c.hora ASC`;
        const [resultados] = await db.query(query, [req.params.clienteId]);
        res.json(resultados);
    } catch (error) {
        res.status(500).json({ error: 'Error al cargar citas' });
    }
```

3.4.5 Historia de Usuario 7: Consulta de agenda diaria (Rol Barbero)

Una vez que el sistema permite realizar reservas, es imprescindible que el profesional tenga una herramienta para visualizar su jornada laboral. Esta funcionalidad transforma la base de datos en una agenda organizada y eficiente para el barbero.

A. Implementación de la Interfaz (Frontend) Se ha desarrollado un panel específico para el rol de barbero. A diferencia de la vista del cliente, aquí la prioridad es identificar rápidamente quién es el cliente asignado y qué servicio se va a realizar. La interfaz presenta las citas en orden cronológico, permitiendo al barbero anticiparse a su carga de trabajo diaria.



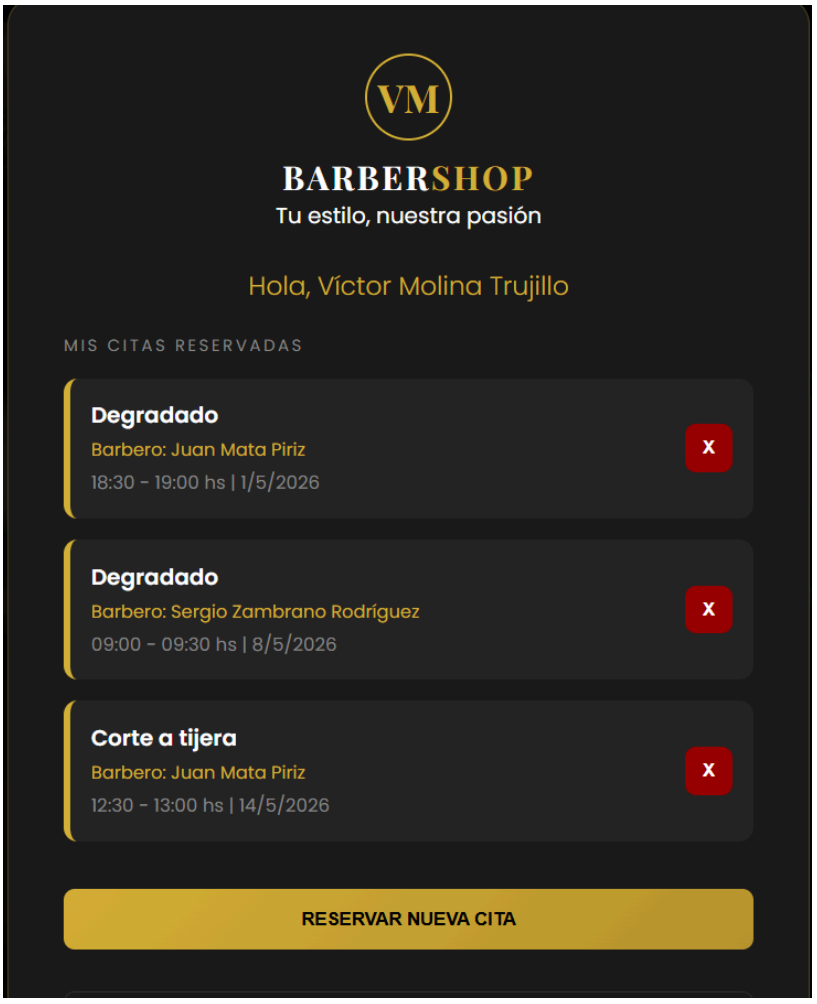
B. Lógica de Consulta Profesional (Backend) La implementación reside en la ruta GET /barbero/:barberoId de citas.js. Los puntos técnicos clave son:

- **Seguridad de Datos**: Al filtrar por el ID del barbero en la cláusula WHERE, aseguramos que cada profesional solo vea su propia agenda, cumpliendo con principios de privacidad de datos.
- **Información Detallada**: Mediante el uso de JOIN, recuperamos el nombre del cliente desde la tabla usuarios y la duración del servicio desde la tabla servicios, permitiendo al barbero gestionar mejor sus tiempos de descanso.

3.4.6 Historia de Usuario 5: Cancelación de citas

Para garantizar la flexibilidad del servicio y la optimización de los huecos horarios, se ha implementado la opción de cancelación autónoma por parte del cliente.

A. Implementación y Flujo de Borrado En la vista de "Mis Citas" del cliente, se ha integrado un botón de cancelación. Técnicamente, este botón activa una petición de tipo **DELETE** hacia el servidor, enviando el identificador único de la cita seleccionada.



B. Impacto en la Persistencia de Datos El servidor procesa esta solicitud mediante el controlador router.delete('/:id'). Al ejecutar la sentencia DELETE FROM citas, el registro se elimina de forma permanente, permitiendo que ese bloque horario vuelva a estar disponible en las consultas de disponibilidad de otros usuarios.

3.5 TERCER SPRINT

Este último sprint se centró en la robustez del sistema. El objetivo fue añadir capas de seguridad lógica para evitar citas incoherentes y mejorar la comunicación con el cliente antes de que la información se guarde definitivamente.

3.5.1 Lógica de Validación y UX

A diferencia de los sprints anteriores, aquí el trabajo se repartió equitativamente entre el frontend y el backend para asegurar que las restricciones fueran infranqueables:

- **Validación de Fechas:** Se implementó lógica en el calendario para deshabilitar días no permitidos (pasados y festivos).
- **Gestión de Estados Temporales:** Se refinaron las consultas SQL para que el tiempo actuara como un filtro automático en la visualización de agendas.

3.5.2 Historias de usuario comprobadas en este sprint

ID	ROL	HISTORIA DE USUARIO	ESTADO
HU-06	Cliente	Como cliente, quiero ver un resumen con los datos de mi reserva antes de confirmarla.	COMPROBADO
HU-08	Cliente	Como cliente, quiero que me impida seleccionar tanto fechas pasadas como los domingos.	COMPROBADO
HU-10	Todos	Como usuario, no quiero que aparezcan en mi agenda citas pasadas.	COMPROBADO

3.5.3 Historia de Usuario 6: Resumen de reserva antes de confirmar

Para evitar errores accidentales y mejorar la usabilidad, se añadió un paso de verificación.

A. Implementación de la Interfaz (Frontend) Antes de realizar la llamada a la API, el sistema despliega un resumen con el nombre del barbero, el servicio, el precio y la hora

elegida. Esto da al cliente una última oportunidad de corregir datos, reduciendo el número de cancelaciones posteriores.



B. Estructura Lógica del Modal (Código HTML) Desde el punto de vista del desarrollo, la funcionalidad se apoya en un contenedor con el ID modalConfirmacion que utiliza estilos de posicionamiento para superponerse a la interfaz principal. Como se observa en la captura de código, se han empleado etiquetas con identificadores específicos (modalResBarbero, modalResServicio, etc.) que actúan como "huecos" de información. Estos elementos son manipulados dinámicamente mediante JavaScript, asignando los valores seleccionados por el usuario justo antes de invocar la función de envío final enviarReserva().

```
<div id="modalConfirmacion" class="modal" style="z-index: 10000; align-items: center; justify-content: center; display: none;">
    <div class="modal-content" style="max-width: 400px; text-align: center;">
        <h3 style="color: #d4af37; margin-top: 0; font-size: 1.2rem; font-family: 'Playfair Display';">¿Confirmar Reserva?</h3>
        <p style="margin-bottom: 20px; font-size: 0.9rem; color: #ccc;">Revisa que los datos sean correctos:</p>

        <div style="text-align: left; margin-bottom: 25px; background-color: #2a2a2a; padding: 15px; border-radius: 5px;">
            <p style="margin: 5px 0;"><strong>Barbero:</strong> <span id="modalResBarbero" style="color: #d4af37;"></span></p>
            <p style="margin: 5px 0;"><strong>Servicio:</strong> <span id="modalResServicio" style="color: #d4af37;"></span></p>
            <p style="margin: 5px 0;"><strong>Fecha:</strong> <span id="modalResFecha" style="color: #d4af37;"></span></p>
            <p style="margin: 5px 0;"><strong>Hora:</strong> <span id="modalResHora" style="color: #d4af37;"></span></p>
        </div>

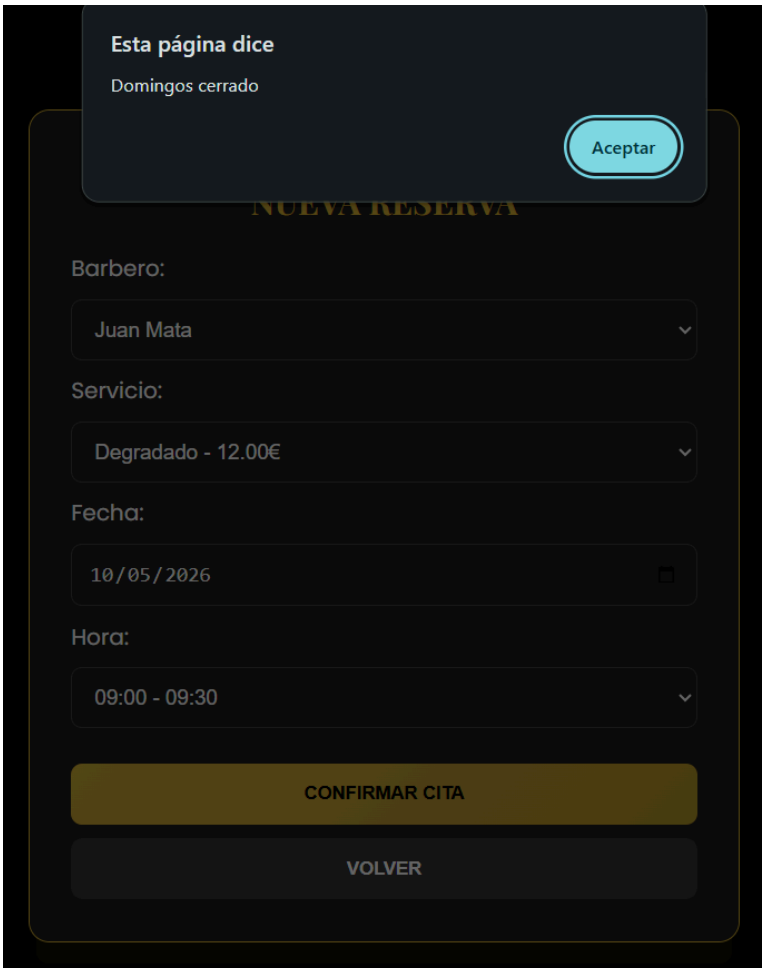
        <div style="display: flex; justify-content: space-between; gap: 15px;">
            <button onclick="cerrarConfirmacion()" style="background-color: #333; color: white; padding: 12px; border: none; border-rad
            <button onclick="enviarReserva()" style="background-color: #d4af37; color: black; padding: 12px; border: none; border-radiu
        </div>
    </div>
</div>
```

3.5.4 Historia de Usuario 8: Validación de calendario (Restricción de pasados y domingos)

Esta funcionalidad garantiza que la base de datos solo reciba registros coherentes con el calendario laboral de la barbería, bloqueando cualquier intento de reserva en días no operativos o fechas anteriores a la actual.

A. Implementación de Restricciones en el Frontend Para ofrecer una experiencia de usuario óptima, las validaciones se ejecutan en tiempo real sobre el elemento <input type="date">. Se han implementado dos capas de control:

1. **Bloqueo de fechas pasadas**: Mediante JavaScript, se calcula la fecha actual y se asigna dinámicamente al atributo min del calendario, impidiendo físicamente la selección de días anteriores.
2. **Exclusión de domingos**: Se ha programado un evento de escucha (change) que analiza el día de la semana seleccionado mediante el método .getDay(). Si el resultado es 0 (domingo), el sistema lanza una alerta informativa y resetea el campo para obligar a una nueva selección.



B. Lógica de validación en JavaScript Como se observa en la captura de código del archivo controlador del frontend, se han implementado estructuras de control para

interceptar selecciones erróneas antes de que estas lleguen a procesarse. El sistema utiliza el objeto Date para comparar la cronología de la cita y el método .getDay() para identificar el descanso dominical (valor 0). En ambos casos, si la validación falla, el sistema interrumpe el flujo mediante un return y resetea el valor del campo, garantizando que solo se realicen peticiones a la API con datos válidos.

```
124         if (fechaCita < hoyCero) {
125             alert("Fecha pasada.");
126             fechaInput.value = '';
127             return;
128         }
129
130         const resS = await fetch('http://localhost:3000/api/servicios');
131         const servicios = await resS.json();
132         const serv = servicios.find(s => s.id == servicioId);
133         const duracion = serv ? serv.duracion : 30;
134
135         const resO = await fetch(`http://localhost:3000/api/citas/ocupadas?barbero_id=${barberoId}&fecha=${fechaSel}`);
136         const ocupadas = await resO.json();
137
138         const dia = fechaCita.getDay();
139
140         if (dia === 0) { alert("Domingos cerrado"); fechaInput.value = ''; return; }
141
```

3.5.5 Historia de Usuario 10: Filtrado de citas pasadas en la agenda

Esta funcionalidad garantiza que tanto el cliente como el barbero dispongan de una interfaz limpia y útil, centrada exclusivamente en los servicios pendientes y futuros. Desde el punto de vista del negocio, evita errores de gestión al no mostrar información que ya no requiere acción.

A. Experiencia de Usuario (UX) y Visualización En los paneles de control de ambos roles, el sistema omite automáticamente cualquier registro cuya fecha sea anterior a la actual. Esto se traduce en una carga de datos más rápida y una navegación más intuitiva, ya que el usuario no tiene que desplazarse entre registros históricos para encontrar su próxima cita.

B. Implementación del Filtro de Caducidad (Frontend)

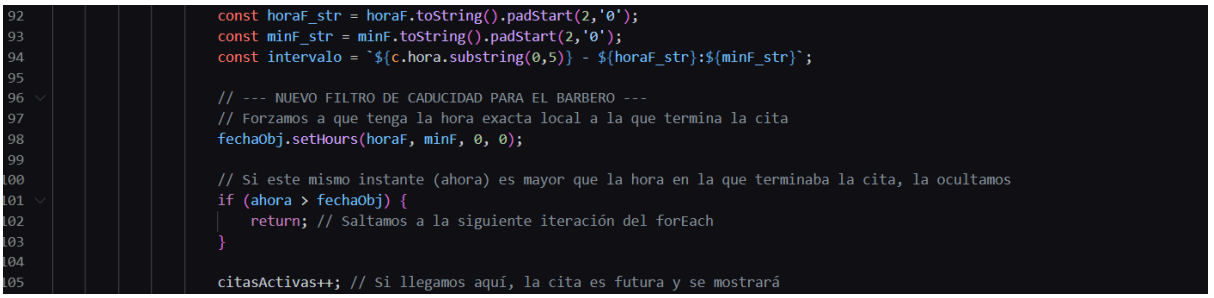
La robustez de la agenda de **VM-Barbershop** reside en que el filtrado no se limita únicamente a la fecha, sino que evalúa el instante preciso en el que una cita deja de ser vigente. Para ello, se ha implementado una lógica de comparación de objetos `Date` en JavaScript que actúa de forma dinámica en ambos perfiles:

- **Filtro para el Barbero:** Como se observa en la captura del controlador del barbero,

```
201             // Calculamos la hora de fin sumando la duración
202             const minFin = (h * 60) + m + duracion;
203             const hFin = Math.floor(minFin / 60);
204             const mFin = minFin % 60;
205             const horaFin = `${hFin.toString().padStart(2,'0')}:${mFin.toString().padStart(2,'0')}`;
206
207             // --- FILTRO DE CADUCIDAD (CORREGIDO) ---
208             // Inicializamos la fecha basándonos en el día guardado en la DB
209             const fechaCitaObj = new Date(c.fecha);
210
211             // Forzamos a que tenga la hora exacta local a la que termina la cita
212             fechaCitaObj.setHours(hFin, mFin, 0, 0);
213
214             // Si este mismo instante (ahora) es mayor que la hora en la que terminaba la cita, la ocultamos
215             if (ahora > fechaCitaObj) {
216                 return;
217             }
218
219             citasActivas++; // Sumamos una a las citas que sí se van a mostrar
220
```

el sistema calcula la hora exacta de finalización de la cita sumando la duración del servicio a la hora de inicio. Mediante la estructura if (ahora > fechaObj), el sistema oculta automáticamente la cita en el momento en que el servicio expira, permitiendo que el barbero vea siempre su carga de trabajo actual y futura.

- **Filtro para el Cliente:** De igual manera, en el panel del cliente,



```
const horaF_str = horaF.toString().padStart(2,'0');
const minF_str = minF.toString().padStart(2,'0');
const intervalo = `${c.hora.substring(0,5)} - ${horaF_str}:${minF_str}`;

// --- NUEVO FILTRO DE CADUCIDAD PARA EL BARBERO ---
// Forzamos a que tenga la hora exacta local a la que termina la cita
fechaObj.setHours(horaF, minF, 0, 0);

// Si este mismo instante (ahora) es mayor que la hora en la que terminaba la cita, la ocultamos
if (ahora > fechaObj) {
    return; // Saltamos a la siguiente iteración del forEach
}

citasActivas++; // Si llegamos aquí, la cita es futura y se mostrará
```

se ha integrado un "Filtro de Caducidad Corregido". Esta lógica asegura que el cliente no visualice citas que ya han concluido, manteniendo su área personal libre de registros obsoletos y centrando su atención en sus próximos compromisos.

Esta doble validación en el frontend garantiza una experiencia de usuario (UX) limpia y profesional, evitando que la interfaz muestre información caducada que podría inducir a errores en la gestión de la barbería.

4. Manual de Instalación y Configuración

4.1 Requisitos del Sistema

Para la ejecución de **VM-Barbershop**, es necesario disponer del siguiente software instalado:

- **Node.js** (Versión 16 o superior).
- **XAMPP / MySQL Server 8.0**.
- **NPM** (incluido con Node.js).

4.2 Pasos para la Instalación

1. **Preparación de la Base de Datos**: Abrir el panel de control de XAMPP e iniciar el módulo de MySQL. Importar el archivo .sql proporcionado para crear las tablas y los registros de prueba (servicios y barbero).

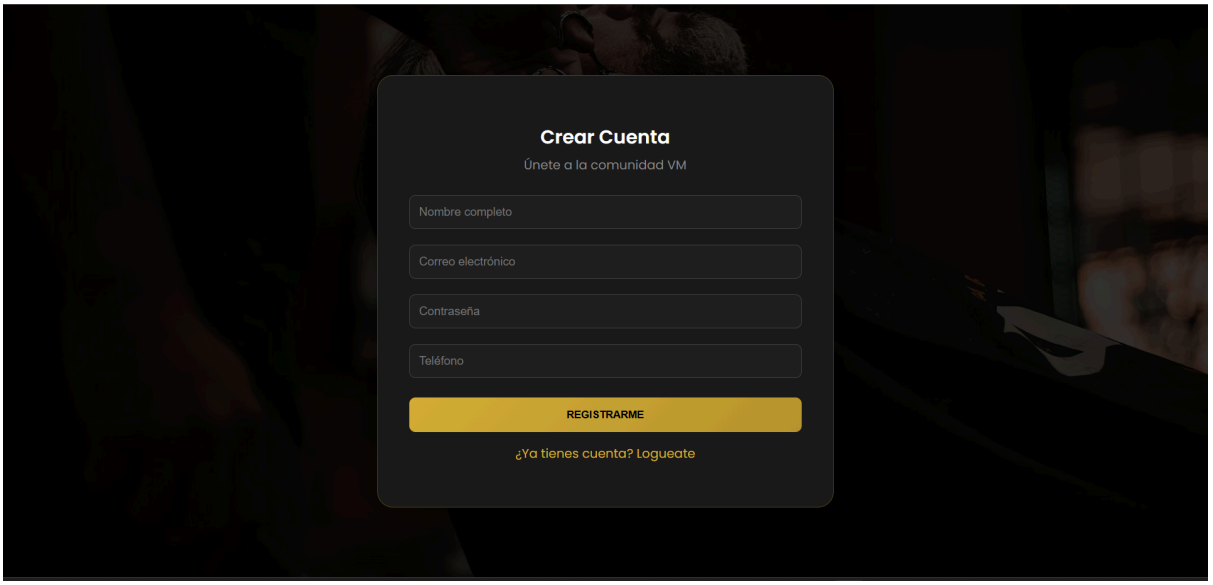
2. **Configuración del Backend**: Acceder a la carpeta backend/ mediante la terminal y ejecutar el comando: npm install Esto descargará todas las dependencias necesarias (express, mysql2, bcryptjs, etc.).
3. **Ajuste de Conexión**: Verificar en el archivo db.js que las credenciales de acceso a MySQL coincidan con las de tu entorno local (host, user, password).
4. **Lanzamiento del Servidor**: Ejecutar el comando: npm start o node server.js El servidor quedará a la escucha en el puerto **3000**.

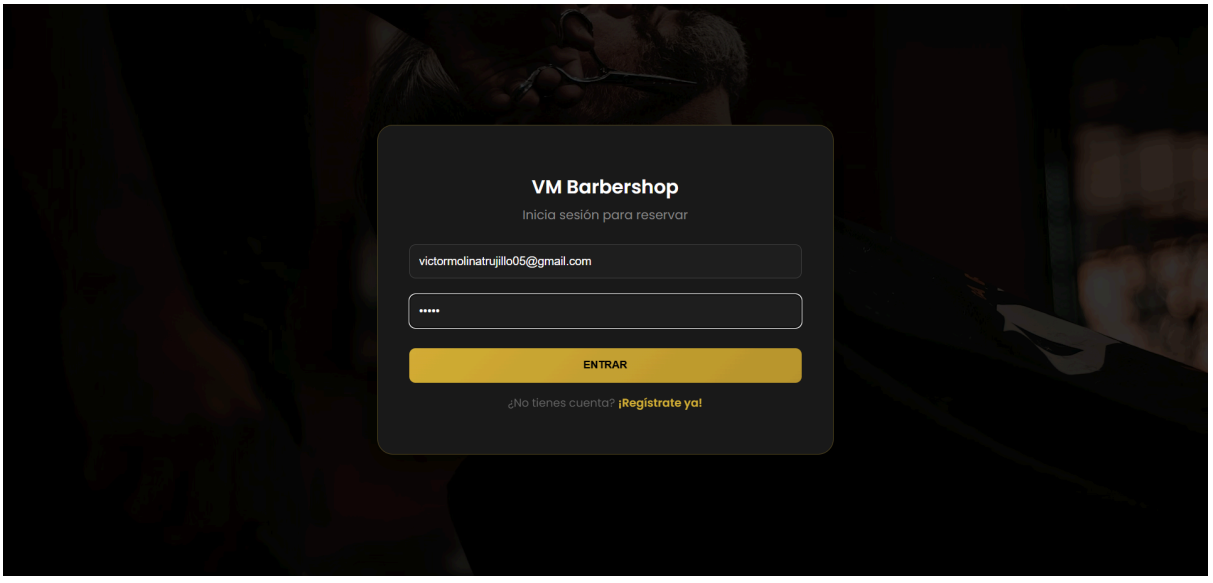
---

5. Manual de Usuario: Guía de Uso para el Cliente

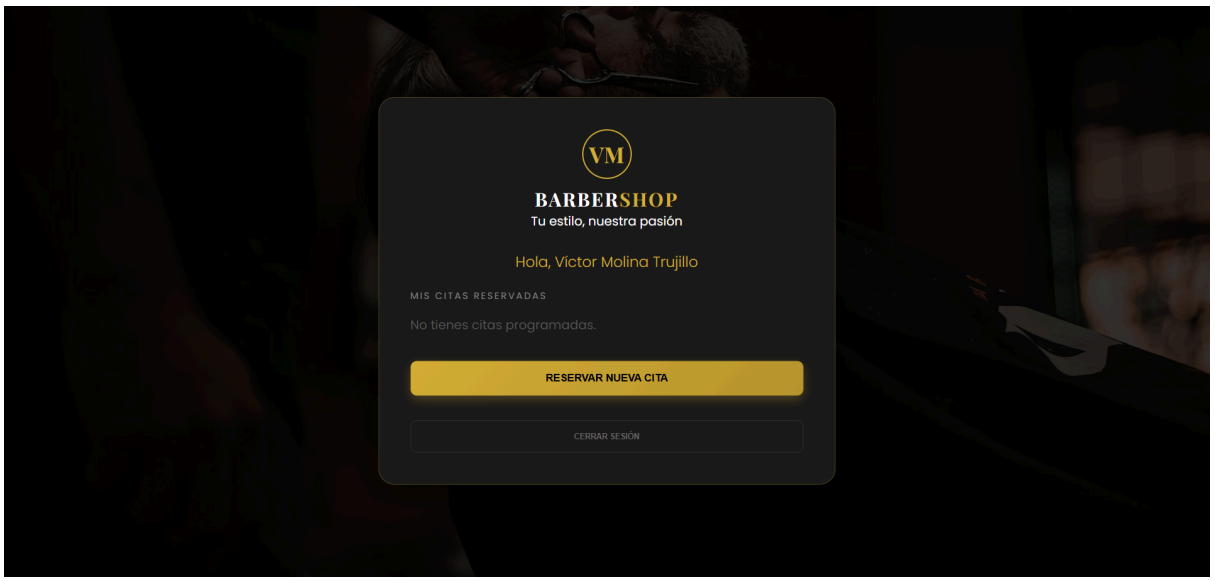
Aquí explicaremos de forma visual cómo un usuario interactúa con la plataforma.

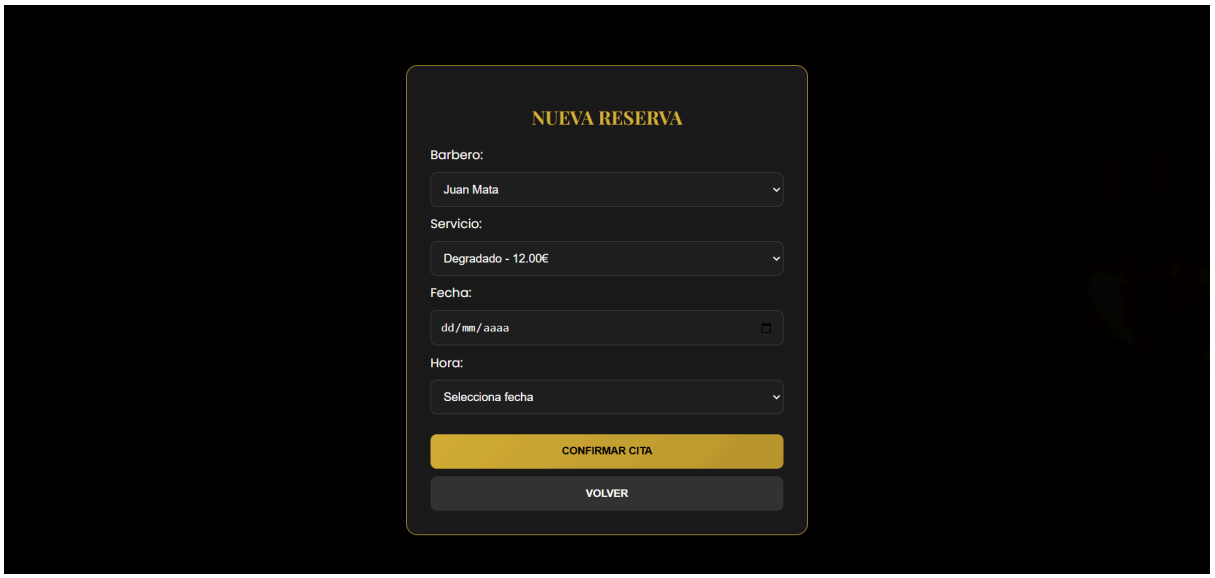
1. **Acceso y Registro**: El usuario comienza creando una cuenta desde el formulario de registro. Una vez registrado, accede mediante el login.



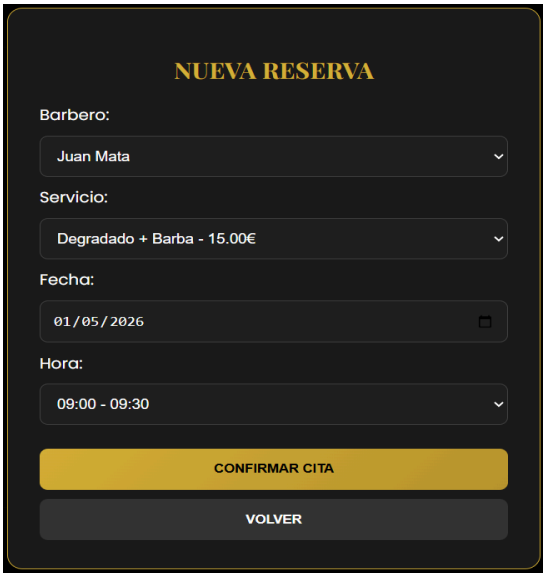


2. **Panel de Nueva Reserva**: Dentro de su área personal, el cliente pulsa en



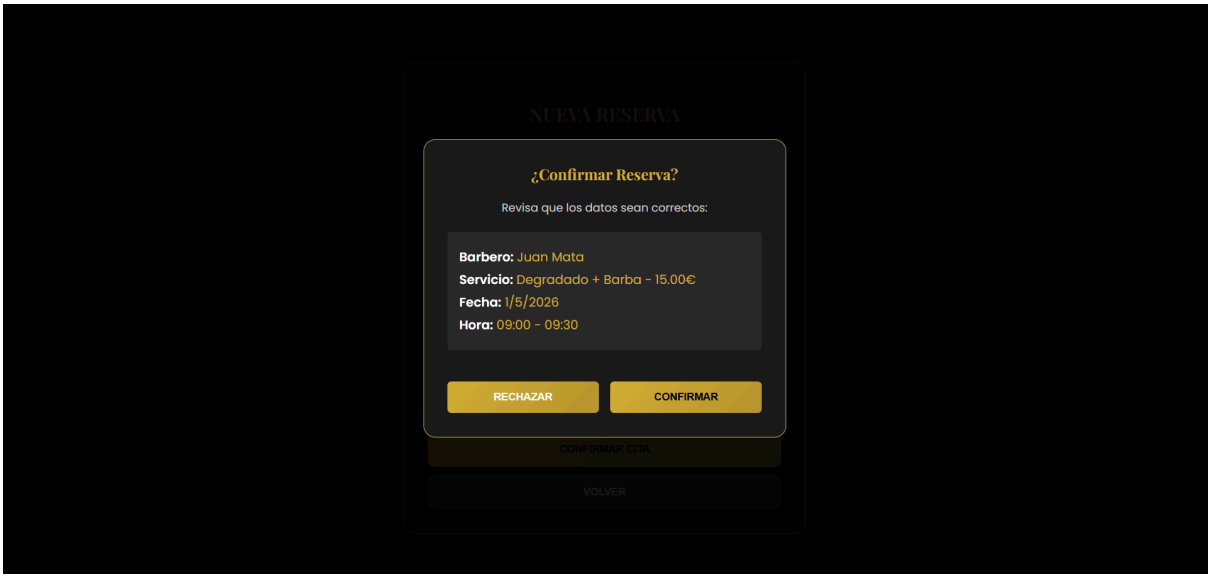


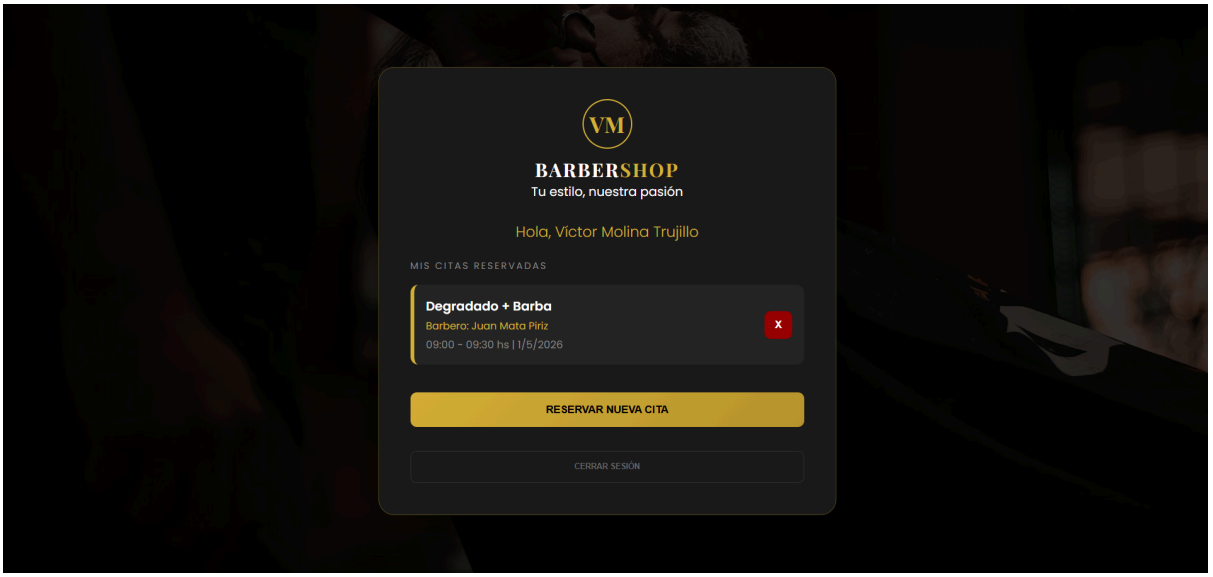
3. **Selección de Datos**: Selecciona el barbero de su preferencia, el servicio (donde verá precios y duraciones) y el día en el calendario.



4. **Gestión de Horarios**: El sistema mostrará únicamente las horas que no estén ocupadas por otros clientes o por servicios previos.

5. **Confirmación**: Tras revisar los datos en el resumen (modal), la reserva se hace efectiva y aparece automáticamente en su listado de "Mis Citas".





6. Conclusiones y Trabajo Futuro

6.1 Conclusiones

El desarrollo de **VM-Barbershop** ha permitido alcanzar un **Producto Mínimo Viable (MVP)** que cumple con el objetivo principal: eliminar la gestión manual de citas y las interrupciones telefónicas. Se ha logrado una arquitectura limpia basada en **MVC**, garantizando una comunicación fluida entre el frontend y el backend. Técnicamente, el proyecto demuestra la capacidad de gestionar bases de datos relacionales, seguridad criptográfica y lógica de negocio compleja en JavaScript.

6.2 Trabajo Futuro

Aunque el sistema es totalmente funcional, se han identificado vías de mejora para futuras versiones:

- **Notificaciones**: Integrar una API de mensajería (como Twilio o WhatsApp) para enviar recordatorios automáticos de cita.
- **Pasarela de Pago**: Implementar el pago online mediante **Stripe** para reducir el absentismo ("no-shows").
- **Despliegue Cloud**: Migrar el entorno local a un servidor en la nube (AWS o Azure) para acceso global.

---

7. Bibliografía y Recursos

- **Node.js Documentation**: https://nodejs.org/docs
- **Express.js Framework**: https://expressjs.com
- **MySQL 8.0 Reference Manual**: https://dev.mysql.com/doc/
- **MDN Web Docs (JavaScript/HTML/CSS)**: https://developer.mozilla.org/
- **Bcryptjs Library**: Documentación en NPM: https://www.npmjs.com/